MODUL PRAKTIKUM 1

Tipe Data, Abstract Data Type & Operasi CRUD

Dosen Pengampu	Dr. Made Ayu Dusea Widyadara, M.Kom
Sub-CPMK	Sub-CPMK1 konsep dasar struktur data, klasifikasi, dan ADT (Bobot 15%)

A. Tujuan Praktikum

- Mahasiswa mampu menerapkan tipe data primitif dan non-primitif Python, termasuk konversi (casting) antar tipe.
- Mahasiswa mampu membedakan spesifikasi Abstract Data Type (ADT) dari implementasinya dalam kode Python.
- Mahasiswa mampu menerapkan operasi dasar Create, Read, Update, Delete (CRUD) pada berbagai struktur data Python.
- Mahasiswa mampu merancang sendiri solusi program dari sebuah soal cerita, tidak sekadar melengkapi kode yang sudah tersedia.

B. Alat dan Bahan

Kebutuhan	Keterangan
Python 3.10+	Diunduh dari python.org, dengan opsi "Add Python to PATH" dicentang saat instalasi
Visual Studio Code	Diunduh dari code.visualstudio.com, dilengkapi ekstensi Python (Microsoft)
Folder kerja	Buat folder khusus, mis. structur-data/pertemuan2/, untuk menyimpan seluruh file latihan

C. Dasar Teori Ringkas

C.1 Tipe Data & Konversi Tipe

Python mengenal tipe data primitif (int, float, str, bool) yang menyimpan satu nilai tunggal, dan tipe data non-primitif (list, tuple, dict, set) yang menyimpan kumpulan nilai. Nilai dapat dikonversi dari satu tipe ke tipe lain menggunakan fungsi bawaan int(), float(), str(), dan bool() — proses ini disebut type casting.

```
umur_teks = "20"
umur = int(umur_teks)       # "20" -> 20
print(type(umur))          # <class 'int'>
```

C.2 Abstract Data Type (ADT)

ADT adalah model logika yang mendefinisikan data dan operasi yang diizinkan tanpa menentukan cara penyimpanannya. Contoh: ADT Stack hanya menjanjikan operasi push, pop, peek, dan is_empty dengan aturan LIFO (Last In, First Out) — bagaimana cara mengimplementasikannya (dengan list, array, atau linked list) adalah persoalan terpisah.

C.3 Operasi CRUD

CRUD adalah singkatan dari Create (menambah data), Read (membaca data), Update (mengubah data), dan Delete (menghapus data). Keempat operasi ini adalah operasi paling dasar yang akan selalu ditemui pada setiap struktur data sepanjang mata kuliah ini.

C.4 Statis vs Dinamis

Struktur data statis memiliki ukuran tetap sejak dideklarasikan (mis. array pada bahasa C). Struktur data dinamis dapat berubah ukurannya saat program berjalan — list Python adalah contoh struktur dinamis: method append() dan pop() mengubah ukurannya secara otomatis.

D. Cara Menggunakan Modul Ini

Setiap latihan pada modul ini terdiri dari dua bagian:

- Contoh Soal — sebuah soal yang mirip/terkait, dibahas dan diselesaikan tuntas beserta kodenya. Baca dan pahami pola penyelesaiannya, boleh dicoba-jalankan sebagai latihan, namun TIDAK dikumpulkan.
- Soal Latihan — soal sesungguhnya yang WAJIB dikerjakan dan dikumpulkan. Tidak disediakan kode awal — susun program dari nol menggunakan pola yang telah dipelajari dari Contoh Soal.

1. Buka VS Code, lalu buka folder kerja praktikum melalui menu File > Open Folder.
2. Baca Contoh Soal tiap latihan, ketik ulang & jalankan kodenya untuk memahami polanya (opsional, tidak dinilai).
3. Kerjakan Soal Latihan pada file terpisah: latihan1.py, latihan2.py, latihan3.py, latihan4.py — inilah yang dikumpulkan.
4. Cocokkan hasil programmu dengan "Format Output yang Diharapkan" pada tiap Soal Latihan untuk verifikasi mandiri.

LATIHAN 1
Tipe Data & Konversi Tipe

CONTOH SOAL (bahan referensi — tidak dikumpulkan)

Sebuah kios pulsa mencatat transaksi penjualan. Nomor HP pelanggan dan nominal pulsa yang dibeli dimasukkan pengguna sebagai teks melalui input(). Buat program yang mengonversi nominal pulsa menjadi angka, menghitung harga jual (nominal + biaya admin Rp1.500), lalu menampilkan hasilnya beserta tipe data (type()) dari nominal sebelum dan sesudah dikonversi.

Pembahasan

Hasil input() selalu bertipe str, sehingga nominal_teks harus dikonversi ke int sebelum dijumlahkan dengan biaya admin. Nomor HP tetap disimpan sebagai str karena tidak akan dihitung secara matematis, dan boleh diawali angka 0.

```
nomor_hp = input("Nomor HP: ")
nominal_teks = input("Nominal pulsa (Rp): ")

nominal = int(nominal_teks)
biaya_admin = 1500
harga_jual = nominal + biaya_admin

print(f"Nomor HP: {nomor_hp}")
print(f"Sebelum konversi: {nominal_teks!r} ({type(nominal_teks)})")
print(f"Sesudah konversi: {nominal} ({type(nominal)})")
print(f"Harga jual: Rp{harga_jual}")
```

SOAL LATIHAN (wajib dikerjakan & dikumpulkan)

Buat program konversi suhu. Program meminta pengguna memasukkan suhu dalam derajat Celsius (input berupa teks). Program harus:

1. Mengonversi input menjadi tipe numerik yang sesuai (boleh mengandung angka desimal).
2. Menghitung nilai suhu yang sama dalam Fahrenheit ($F = C \times 9/5 + 32$) dan Kelvin ($K = C + 273.15$).
3. Menampilkan ketiga nilai suhu dengan dua angka di belakang koma, serta menampilkan type() dari input sebelum dan sesudah konversi.

Ketentuan: gunakan input(), lakukan konversi tipe data secara eksplisit, tanpa library eksternal (mis. math atau numpy).

Format Output yang Diharapkan (contoh, jika input 30)

```
Masukkan suhu (Celsius): 30
Sebelum konversi: '30' (<class 'str'>)
Sesudah konversi: 30.0 (<class 'float'>)
Celsius   : 30.00 C
Fahrenheit: 86.00 F
Kelvin    : 303.15 K
```

Pertanyaan Reflektif

1. Apa yang terjadi jika suhu yang dimasukkan mengandung koma (misalnya "30.5")? Mengapa perlu memilih float, bukan int, untuk kasus ini?
2. Bagaimana seharusnya program merespons jika pengguna memasukkan teks yang bukan angka?

LATIHAN 2
Operasi CRUD pada List

CONTOH SOAL (bahan referensi — tidak dikumpulkan)

Sebuah aplikasi pengingat sederhana menyimpan daftar tugas harian menggunakan list. Buat program yang menambahkan 3 tugas awal, menampilkan seluruh tugas beserta nomor urutnya, mengganti salah satu tugas karena rencana berubah, lalu menghapus satu tugas yang sudah selesai dikerjakan.

Pembahasan

Create memakai append(); Read memakai enumerate() agar nomor urut ikut tampil; Update mengubah elemen lewat index; Delete memakai remove() karena tugas dihapus berdasarkan isinya, bukan posisinya.

```
tugas = []
tugas.append("Beli bahan baku")
tugas.append("Rekap laporan mingguan")
tugas.append("Balas email pelanggan")

for i, t in enumerate(tugas, start=1):
    print(f"{i}. {t}")

tugas[1] = "Rekap & kirim laporan mingguan"   # Update
tugas.remove("Balas email pelanggan")         # Delete

print("Sisa tugas:")
for i, t in enumerate(tugas, start=1):
    print(f"{i}. {t}")
```

SOAL LATIHAN (wajib dikerjakan & dikumpulkan)

Sebuah apotek mengelola antrean pelanggan menggunakan list bernama antrean. Buat program yang:

1. Meminta pengguna memasukkan nama 4 pelanggan satu per satu (gunakan perulangan dan input()), lalu menambahkannya ke antrean.
2. Menampilkan seluruh antrean beserta nomor urutnya.
3. Pelanggan pada urutan ke-2 membatalkan antreannya dan digantikan oleh pelanggan lain — perbarui data tersebut.
4. Pelanggan pertama sudah selesai dilayani — keluarkan pelanggan tersebut dari antrean, sekaligus tampilkan namanya.
5. Tampilkan sisa antrean setelah seluruh proses di atas.

Format Output yang Diharapkan (contoh)

```
Masukkan nama pelanggan ke-1: Sari
Masukkan nama pelanggan ke-2: Budi
Masukkan nama pelanggan ke-3: Rudi
Masukkan nama pelanggan ke-4: Tini
Antrean awal:
1. Sari
2. Budi
3. Rudi
4. Tini
Pelanggan urutan ke-2 digantikan oleh: Wati
Pelanggan yang selesai dilayani: Sari
Sisa antrean:
1. Wati
2. Rudi
3. Tini
```

Pertanyaan Reflektif

1. Mengapa menerima 4 nama pelanggan lebih tepat menggunakan perulangan dibanding menulis input() empat kali secara terpisah?
2. Apa yang perlu diperhatikan jika pelanggan yang "selesai dilayani" ternyata tidak selalu berada di posisi pertama antrean?

LATIHAN 3
ADT Sederhana — Stack dengan List

CONTOH SOAL (bahan referensi — tidak dikumpulkan)

Sebuah peramban (browser) sederhana menyimpan riwayat halaman yang dikunjungi menggunakan prinsip Stack, agar tombol "kembali" selalu menuju halaman yang terakhir dibuka. Buat program yang mengimplementasikan ADT Stack (push, pop, peek, is_empty) untuk mensimulasikan pengunjungan 3 halaman web, lalu menekan tombol kembali satu kali.

Pembahasan

push(url) dipanggil setiap membuka halaman baru (append()). Tombol kembali berarti pop() mengeluarkan halaman teratas (halaman saat ini), sehingga yang tersisa di atas adalah halaman sebelumnya. peek() dipakai untuk melihat halaman aktif tanpa mengeluarkannya dari riwayat.

```
riwayat = []

def push(url):
    riwayat.append(url)

def pop():
    return riwayat.pop() if not is_empty() else None

def peek():
    return riwayat[-1] if not is_empty() else None

def is_empty():
    return len(riwayat) == 0

push("beranda.com")
push("beranda.com/produk")
push("beranda.com/produk/detail")

halaman_ditinggalkan = pop()   # tombol kembali ditekan
print(f'Meninggalkan halaman: {halaman_ditinggalkan}')
print(f'Halaman aktif sekarang: {peek()}')
print(f'Riwayat kosong? {is_empty()}')
```

SOAL LATIHAN (wajib dikerjakan & dikumpulkan)

Sebuah warung makan mencuci piring dan menumpuknya di satu tumpukan (stack) sebelum dipakai kembali. Buat program yang mengimplementasikan ADT Stack (push, pop, peek, is_empty) untuk kasus berikut:

1. Tiga piring bersih ditumpuk satu per satu: "Piring A", "Piring B", "Piring C".
2. Pelayan mengambil satu piring paling atas untuk dipakai — tampilkan piring mana yang terambil.

3. Setelah itu, satu piring bersih baru ("Piring D") ditambahkan ke tumpukan.
4. Tampilkan piring yang berada paling atas tumpukan saat ini, tanpa mengambilnya.
5. Ambil semua piring dari tumpukan satu per satu (gunakan is_empty() sebagai kondisi perulangan while) sambil menampilkan urutan piring yang terambil, sampai tumpukan benar-benar kosong.

Format Output yang Diharapkan (contoh)

```
Piring yang terambil pelayan: Piring C
Piring paling atas sekarang: Piring D
Mengambil semua piring:
- Piring D
- Piring B
- Piring A
Tumpukan piring sudah kosong.
```

Pertanyaan Reflektif

1. Pada langkah 5, mengapa urutan piring yang terambil adalah D, B, A — bukan urutan piring saat ditumpuk (A, B, C, D)?
2. Sebutkan satu situasi lain (di luar contoh) dalam kehidupan sehari-hari yang juga mengikuti prinsip LIFO seperti tumpukan piring ini.

LATIHAN 4
Studi Kasus — Basis Data Sederhana (List of Dict)

CONTOH SOAL (bahan referensi — tidak dikumpulkan)

Sebuah perpustakaan kecil mencatat data buku menggunakan list berisi dict, dengan setiap dict memiliki kunci "judul" dan "stok". Buat program yang melakukan operasi CRUD lengkap pada data tersebut, serta menghitung total seluruh stok buku yang tersedia.

Pembahasan

Setiap buku disimpan sebagai satu dict di dalam list. Read pada field tertentu memakai buku[i]["judul"]; Update mengubah nilai pada key tertentu; Delete memakai pop(index); dan menghitung total stok memerlukan traversal (perulangan) sambil mengakumulasi nilai stok tiap buku.

```
buku = []
buku.append({"judul": "Belajar Python", "stok": 5})
buku.append({"judul": "Struktur Data Dasar", "stok": 3})

print(buku)                  # Read semua
print(buku[0]["judul"])      # Read satu field

buku[1]["stok"] = 4          # Update
buku.pop(0)                  # Delete

def total_stok(daftar_buku):
    total = 0
    for b in daftar_buku:
        total += b["stok"]
    return total

print("Total stok:", total_stok(buku))
```

SOAL LATIHAN (wajib dikerjakan & dikumpulkan)

Sebuah toko tanaman hias mencatat data produknya menggunakan list berisi dict, dengan setiap dict memiliki kunci "nama" dan "harga" (dalam Rupiah). Buat program yang:

1. Menambahkan (Create) 4 data produk ke dalam list bernama produk.
2. Menampilkan (Read) seluruh data produk, lalu menampilkan nama produk termahal — dicari secara manual dengan perulangan & perbandingan, tanpa fungsi max().
3. Mengubah (Update) harga salah satu produk karena ada penyesuaian harga.
4. Menghapus (Delete) satu produk yang sudah tidak dijual lagi.
5. Membuat fungsi total_nilai_stok(daftar_produk) yang menjumlahkan seluruh harga produk yang tersisa, lalu menampilkan hasilnya.

Format Output yang Diharapkan (contoh)

```
Daftar produk:
- Monstera: Rp150000
- Aglaonema: Rp85000
- Sirih Gading: Rp35000
- Kaktus Mini: Rp25000
Produk termahal: Monstera (Rp150000)
Setelah update & hapus:
- Aglaonema: Rp95000
- Sirih Gading: Rp35000
- Kaktus Mini: Rp25000
Total nilai stok: Rp155000
```

Pertanyaan Reflektif

1. Mengapa mencari produk termahal secara manual (loop + perbandingan) penting dipelajari, meskipun Python punya cara lebih singkat?
2. Bagaimana strukturmu perlu berubah jika setiap produk juga mencatat jumlah stok (bukan hanya harga satuan), agar total_nilai_stok() menghitung harga x jumlah stok?

E. Ketentuan Pengumpulan

- Kumpulkan 4 file Soal Latihan: latihan1.py, latihan2.py, latihan3.py, latihan4.py (kode Contoh Soal tidak perlu dikumpulkan).
- Sertakan tangkapan layar (screenshot) hasil eksekusi tiap file di terminal VS Code.
- Kumpulkan sebelum Pertemuan 3 (Rabu, 30 September 2026) melalui kanal yang ditentukan .